

SmartCare v0.1 - Initial Engineering Brief and AI Activity Card

Week 4 | Part C (Problem framing + AI Ask-Check-Explain workbook, 1.0 mark)

Student name	Mithun Naveen
Student ID	U3324934
Tutorial group	
Submission	Week 4 - Stage 1

Scenario: SmartCare Community Clinic currently uses spreadsheets and paper records to manage patients and appointments. The client says: 'We need software to help manage patients, practitioners and appointments.' This is not yet a complete specification.

1. Problem summary

SmartCare is a small community clinic where GPs consult patients. Patient information and bookings are stored in spreadsheets, paper records and manual systems, resulting in duplicate bookings, difficulties locating patient records, inconsistent appointment status, lack of visibility of practitioner availability, manual cancellations, no reliable appointment history and inability to generate basic reports. Management is looking for an application that is easy to maintain and use, and not a hospital information system, that will first support patient, practitioner and appointment management. The brief is still not complete: the users, the data, the rules and the measures of success are not confirmed, so the first engineering step is to clarify the scope with the client before designing anything.

2. Initial stakeholders

Stakeholder	Possible need
Receptionist / admin staff	Find a patient quickly; book and cancel without duplicates; see accurate status.
Practitioner (GP)	View own schedule and availability; trust appointment data.
Patient	Dependable bookings; records found quickly; personal data kept private.
Clinic manager / owner	Reliable history and simple reports; low-complexity, maintainable system.
IT support / maintainer	Readable, testable code; ability to extend later.

3. Initial features

Feature	Confirmed or provisional?	Why?
Register patients and search/find patient records	Confirmed	Client explicitly wants to manage patients; 'difficulty locating patient records' is a stated problem.
Book appointments (patient + practitioner + time)	Confirmed	Core of the client statement ('manage ... appointments').
Prevent duplicate bookings	Confirmed	Duplicate bookings are the first listed operational problem. The exact definition of 'duplicate' is still provisional.
Cancel appointments and keep a reliable history	Confirmed	'Manual cancellation processes' and 'lack of reliable appointment history' are stated problems. Reason/late-cancel rules are unknown.
Practitioner availability / schedule view	Provisional	'Limited visibility of availability' is stated, but it is unclear whether availability means working hours, free slots or booked appointments.

Feature	Confirmed or provisional?	Why?
Basic operational reports	Provisional	Reports are mentioned, but which reports (counts? per practitioner?) is not specified.

4. Questions for the client

1. What does a 'duplicate' booking mean? Same practitioner and time, same patient twice or both?
2. Who will be using the system and should there be different access for different roles (receptionist, GP, manager)?
3. What information should be kept and what privacy/security rules should the system adhere to?
4. What is the average number of patients, practitioners, and appointments per day? What is the average length of an appointment?
5. Who can cancel, any cut-off time, is a reason recorded, must cancelled appointments stay in the history?

5. What we do not yet know

1. Definition of practitioner availability (working hours, breaks, leave) and who is responsible for this.
2. What constitutes a good report.
3. If data needs to be stored across sessions, shared by multiple staff members simultaneously, or transferred from the current spreadsheets.
4. If any integration (SMS, billing, existing systems) is expected - none has been requested so far.

AI Activity Card - Ask, Check, Explain

Before AI

The code I believe does the following: It maintains three lists, one for patients, one for practitioners, and one for appointments, all of which are dictionaries. `book appointment()` checks if the patient and practitioner ID exist and if the practitioner has an appointment with exactly the same time text, and then adds a new appointment with status 'scheduled'. `cancel appointment()` changes the status to 'cancelled' and leaves the record in the list, which frees the slot up again.

Problems I can already identify: (1) time is free text, so tomorrow is OK; (2) status values are plain strings; (3) cancelling twice is silently allowed; (4) data is stored in global lists, so it is difficult to test and not safe for multiple users; (5) no saving when the program exits.

AI request

Act as a tutor. Explain this code and identify potential problems. Do not complete the whole thing. Ask me questions that help me reason about the solution.

Questions the AI asked me (summarised): What should happen if two receptionists book the same practitioner at once? Does 'the same time' mean identical start time or overlapping time? Who is allowed to cancel, and should a cancelled appointment ever be reopened?

Evaluate

Suggestion (AI, summarised)	Useful	Unclear	Incorrect	Out of scope
S1. Status values are plain strings; a typo such as 'canceled' would silently break the duplicate check.	Yes			
S2. Time is free text, so '09:00' and '9:00' look different and 'tomorrow' is accepted.	Yes			
S3. Cancelling an already cancelled appointment succeeds silently; consider an error.	Yes			
S4. Module-level lists make the code hard to test and unsafe if several users share it.		Partly - depends on how the clinic will deploy it		
S5. Add a SQLite database with password login and role-based access.				Yes - not in the brief
S6. IDs from <code>len(appointments)+1</code> could repeat if appointments were ever deleted.		Yes - no delete exists today		

Decide

Suggestion	Decision	Reason
S1 string status	Accept	Real weakness; replace with constants now and an enum in Stage 4.
S2 free-text time	Accept	Reproduced with a test; v0.2 parses times with datetime (FR-05, BR-05).
S3 silent double cancel	Accept	Reproduced with a test; becomes requirement FR-10 (illegal transitions rejected).
S4 global state	Modify	Accept the testability point; the concurrency/deployment question goes to the client list, not the code.
S5 database + login	Reject	No client evidence. Recorded as an open question (persistence, access control).
S6 ID generation	Keep unverified	Cannot occur today (cancel keeps records). Revisit if deletion is ever requested.

Verify

Check	Done?	Evidence
Run the code	Yes	evidence/demo_output.txt - booking, duplicate rejection, cancel, rebook all behave as described.
Test normal input	Yes	test_starter_prototype.py - 4 normal-input tests pass (search, book, duplicate rejected, cancel frees slot).
Test unusual input	Yes	5 unusual-input tests pass: unknown patient and blank name are rejected; 3 'known limitation' tests prove the S2/S3 weaknesses are real (free-text time, 09:00 vs 9:00, silent double cancel).
Compare with requirements	Yes	Brief problem list: duplicate prevention present; history partly (cancel keeps records); schedule view/reports not yet built.
Check documentation	Planned	Read the Python datetime.strptime documentation
Ask tutor/peer	Planned	Bring the 'what is a duplicate booking?' question to the Week 5 tutorial.

Explain

Can I explain the final code without reading the AI response? Yes. The program is a dictionary-based prototype: each record is a dict, booking walks the appointment list to look for a scheduled appointment for the same practitioner at the same time string, and cancelling just changes the status, but does not delete. I can also tell you why it is weak: strings and free text can have typos and ambiguous times, and there's nothing to prevent an illegal cancel. What else do I need to know? What the clinic means by 'duplicate' and 'availability', and how to model appointment states properly; these are the focus of Stages 2-4.